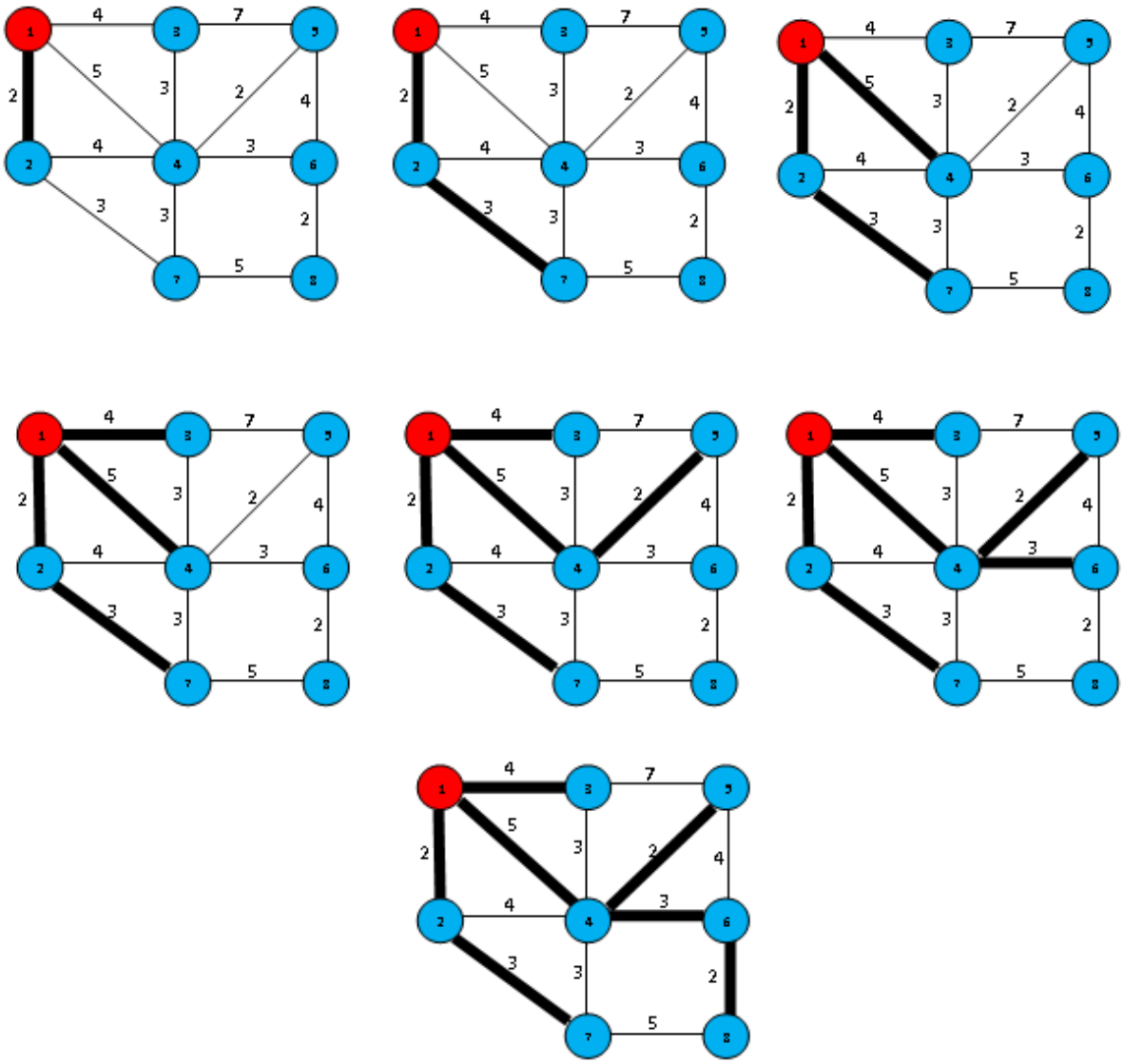
2.3 Shortest Path Algorithms

Shortest Path Algorithms are used to find the minimum-cost path between vertices in a graph. They are widely used in computer networks, GPS navigation, maps, routing protocols, AI, and data science.

In a weighted graph, the shortest path between two vertices is the path whose sum of edge weights is minimum.

- Weight → distance, cost, time, or effort
- Source vertex → starting point
- Destination vertex → ending point

Dijkstra's Algorithm



Idea

- Always chooses the nearest unvisited vertex
- Greedy approach

Conditions

- Works only for non-negative edge weights
- Can be directed or undirected graph

Steps

1. Set distance of source = 0, others = ∞
2. Pick vertex with minimum distance
3. Update distances of its neighbors
4. Repeat until all vertices are visited

Time Complexity

- Using priority queue: O((V + E) log V)

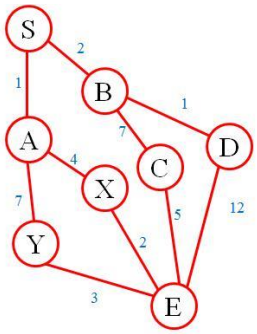
Applications

- GPS navigation
- Network routing (OSPF)
- Maps and path planning

Limitation

Cannot handle negative edge weights

*A (A-Star) Algorithm**



- Values for h:
  A:5, B:6, C:4, D:15, X:5, Y:8

Expand S
{S,A} *f*=1+5=6
{S,B} *f*=2+6=8

Expand A
{S,B} *f*=2+6=8
{S,A,X} *f*=(1+4)+5=10
{S,A,Y} *f*=(1+7)+8=16

Expand B
{S,A,X} *f*=(1+4)+5=10
{S,B,C} *f*= (2+7)+4=13
{S,A,Y} *f*=(1+7)+8=16
{S,B,D} *f*= (2+1)+15=18

Expand X
{S,A,X,E} is the best path... (costing 7)

Idea

- Uses heuristic function (h(n))
- Chooses path based on:

$$f(n) = g(n) + h(n)$$

where

- g(n) = actual cost
- h(n) = estimated cost to goal

Conditions

- Heuristic must be admissible (never overestimate)

Applications

- Game AI
- Robotics
- Map navigation

Advantage

Faster than Dijkstra in many cases

Graph Connectivity

Definition:

Graph connectivity describes whether nodes are reachable from each other.

Types of Connectivity

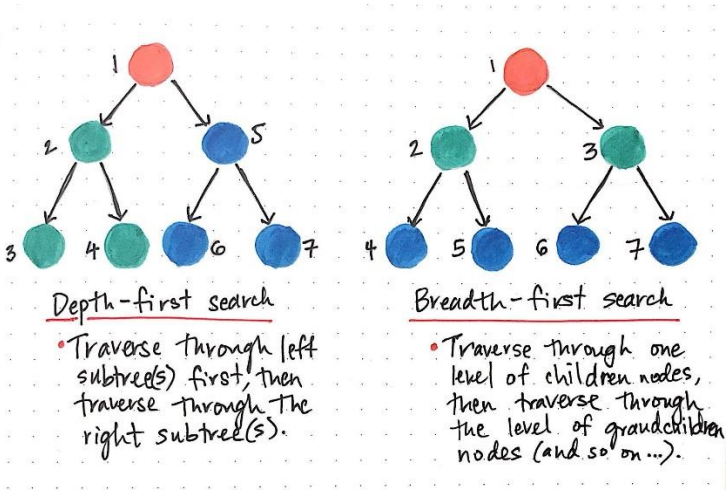
1. Connected Graph – path exists between every pair
2. Disconnected Graph – at least one unreachable node
3. Strongly Connected (Directed Graph)
    - Every node reachable from every other node
4. Weakly Connected
    - Ignoring directions, graph is connected

Importance:

- Detecting isolated users
- Community formation
- Network robustness

Graph Traversal Algorithms

Breadth-First Search (BFS)



Breadth First Search (BFS) is a graph traversal algorithm that explores vertices level by level starting from a given source node.

BFS is used to find the shortest path in an unweighted graph.

Key Characteristics

- Uses a Queue (FIFO)
- Visits all neighbors before moving to the next level
- Works for graphs and trees
- Finds minimum number of edges (hops)

How BFS Works (Step-by-Step)

1. Start from the source vertex
2. Mark it as visited
3. Insert it into the queue
4. While queue is not empty:
    - Remove a vertex from queue
    - Visit all its unvisited adjacent vertices
    - Mark them visited and add to queue

BFS Algorithm (Pseudo Code)

```
BFS(Graph, start):
    create empty queue Q
    mark start as visited
    enqueue start into Q
    while Q is not empty:
        v = dequeue Q
        print v
        for each neighbor u of v:
            if u not visited:
                mark u visited
                enqueue u into Q
```

Time and Space Complexity

- Time Complexity: O(V + E)
- Space Complexity: O(V)

Where:

- V = number of vertices
- E = number of edges

Applications of BFS

✓ Shortest path in unweighted graphs
✓ Social networks (friends-of-friends)
✓ Web crawling
✓ Network broadcasting
✓ AI & game pathfinding
✓ Checking graph connectivity

Depth-First Search (DFS)

Depth First Search (DFS) is a graph traversal algorithm that explores a path as deep as possible before backtracking.

> DFS is commonly used to explore structure, detect cycles, and analyze connectivity in graphs.

Key Characteristics

- Uses Stack (implicitly via recursion or explicitly)
- Goes deep first, then backtracks
- Can be applied to graphs and trees
- Does not guarantee shortest path

How DFS Works (Step-by-Step)

1. Start from the source vertex
2. Mark it as visited
3. Visit one unvisited adjacent vertex
4. Repeat steps 2–3 until no unvisited neighbor
5. Backtrack to explore remaining vertices

Time and Space Complexity

- Time Complexity: O(V + E)
- Space Complexity: O(V)

Where:

- V = vertices
- E = edges

Applications of DFS

✓ Detecting cycles in graph
✓ Topological sorting
✓ Finding connected components
✓ Solving mazes and puzzles
✓ Path existence checking
✓ AI game tree exploration